WEEKLY PYTHON ASSESSMENT

Candidate Preparation & Practice Guide

Learn the concepts. Apply the logic. Practice independently.

Assessment levels	Core areas
Easy and Medium	Collections, text and data processing

For candidates

This guide develops the programming skills required for a practical Python assessment. It includes concept explanations, runnable examples, independent exercises, a seven-day study plan, and a readiness checklist.

Assessment confidentiality: The exercises below are learning examples, not the actual assessment questions. Focus on understanding and independently applying each technique.

1. Core topics to focus on

Python lists: Create lists, loop through values, append items, and keep original order.

Sets and membership: Use set(), add(), in, and not in for efficient lookups; understand that set iteration order should not be relied on.

Loops and conditions: Combine for, if, comparisons, and membership checks to build results.

Functions and return values: Define functions with parameters; return values rather than only printing.

String processing: Use lower(), split(), strip(), and understand case-sensitive comparisons.

Dictionaries and frequency counts: Build key-value mappings, use get(), and increment occurrence counts.

collections.Counter: Import Counter, create frequency counts, and use most_common().

Tuples, sorting and ranking: Work with (item, value) pairs, sorted(), lambda, reverse=True, and slicing.

2. Worked learning examples

Lists, loops and filtering

Use a loop and conditional statement to build a new list.

```
numbers = [10, 20, 30, 40]
result = []
for num in numbers:
    if num > 20:
        result.append(num)
print(result)
```

Output: [30, 40]

Sets and membership

A set is useful when the main goal is checking whether a value exists.

```
registered = {"A101", "A102", "A103"}
new_id = "A104"
if new_id not in registered:
    registered.add(new_id)
print("A104" in registered)
```

Output: True

Functions and return values

Autograders typically check a returned value; printing is not equivalent.

```
def calculate_total(prices):
    total = 0
    for price in prices:
        total += price
```

```
    return total
print(calculate_total([100, 200, 50]))
```

Output: 350

Strings and whitespace

Normalize letter case and split a sentence on whitespace.

```
message = "Python Makes Coding Easier"
words = message.lower().split()
print(words)
```

Output: ['python', 'makes', 'coding', 'easier']

Dictionary-based counting

Dictionaries can count how many times each category occurs.

```
products = ["pen", "book", "pen", "pencil", "book"]
counts = {}
for product in products:
    counts[product] = counts.get(product, 0) + 1
print(counts)
```

Output: {'pen': 2, 'book': 2, 'pencil': 1}

Counter for frequency analysis

The Counter class is a concise way to calculate occurrences.

```
from collections import Counter
items = ["red", "blue", "red", "green", "red"]
frequency = Counter(items)
print(frequency.most_common(1))
```

Output: [('red', 3)]

Sorting tuples by numeric values

Sort paired data using the second element, then select the required portion.

```
sales = [("Laptop", 45), ("Mouse", 80), ("Keyboard", 30)]
ranked = sorted(sales, key=lambda x: x[1], reverse=True)
print(ranked[:2])
```

Output: [('Mouse', 80), ('Laptop', 45)]

3. Independent practice questions

01 / EASY - Identify even transaction numbers

Write get_even_numbers(numbers). Return all even integers in their original sequence.

Sample input: `[7, 12, 9, 18, 21, 24]`

Expected output: `[12, 18, 24]`

Focus: Lists, loops and conditions

02 / EASY - Check event registration

Write is_registered(registered_ids, search_id). Use a set for membership checking; return a Boolean.

Sample input: `[105, 108, 112, 120], search_id=112`

Expected output: `True`

Focus: Sets, membership and return values

03 / EASY - Filter valid product prices

Write valid_prices(prices). Return prices strictly greater than zero, keeping their order.

Sample input: `[250, -10, 500, 0, 125]`

Expected output: `[250, 500, 125]`

Focus: Lists, conditions and order

04 / MEDIUM - Count product categories

Write category_counts(categories). Return a dictionary containing the occurrence count of each category.

Sample input: `["Electronics", "Books", "Books", "Clothing", "Electronics", "Books"]`

Expected output: `{"Electronics": 2, "Books": 3, "Clothing": 1}`

Focus: Dictionaries and frequency counting

05 / MEDIUM - Measure feedback length

Write feedback_length(message). Return the number of whitespace-separated words in a feedback string.

Sample input: `"Python programming is very useful"`

Expected output: `5`

Focus: String splitting and counting

06 / MEDIUM - Rank employees

Write rank_employees(scores, limit). Sort employees by their numeric performance score descending; return up to limit (name, score) tuples. Assume all scores differ.

Sample input: `{"Amit": 82, "Neha": 95, "Ravi": 76, "Priya": 90}, limit=2`

Expected output: `[("Neha", 95), ("Priya", 90)]`

Focus: Dictionaries, tuples, sorting, slicing

4. Additional self-practice

- Return all values above a caller-supplied threshold.
- Check which IDs in one group also appear in another group.
- Convert a sentence to lowercase and display its words.
- Build a dictionary of numeric occurrence counts.
- Use Counter to count characters in a string.
- Sort a list of (name, score) pairs by score.
- Return a selected number of items after sorting.
- Test functions with empty lists and strings.

5. Common mistakes to avoid

- Printing instead of returning the requested value.
- Returning the wrong type (for example, a set instead of a list).
- Sorting when the problem expects an existing sequence to remain unchanged.
- Overlooking uppercase and lowercase normalization requirements.
- Hardcoding sample answers instead of processing actual inputs.
- Ignoring supplied parameters or empty-input behavior.
- Using incorrect function names or forgetting imports.

6. Suggested seven-day study plan

1	Lists and loops	Iterate, filter and append
2	Sets and conditions	Membership checks and sets
3	Functions	Parameters, return values and test inputs
4	Strings and dictionaries	Transform text and build counts
5	Counter and sorting	Frequency, tuples and ranking
6	Practice problems	Solve all six independently
7	Revision and mock	Review edge cases and weak areas

7. Final readiness checklist

☐ Write parameterized functions that return outputs

- ☐ Process lists with loops and conditions
- ☐ Use sets and membership operators
- ☐ Apply lower(), strip() and split()
- ☐ Count items with dictionaries and Counter
- ☐ Work with tuples, sorted() and slicing
- ☐ Preserve order when it is required
- ☐ Match exact output data types
- ☐ Test empty and varied input values
- ☐ Avoid hardcoded answers

Final guidance: Understand the logic, write your own solutions, and verify them with multiple inputs. The actual assessment may use different scenarios and data.